# Process Outline: Metrology Algorithms Challenge

Levi Hauser

March 20, 2026

*[Note: This document is an outline of my problem-solving process and my solution to the challenge. For documentation of the code, please see* `README.md`*.]*

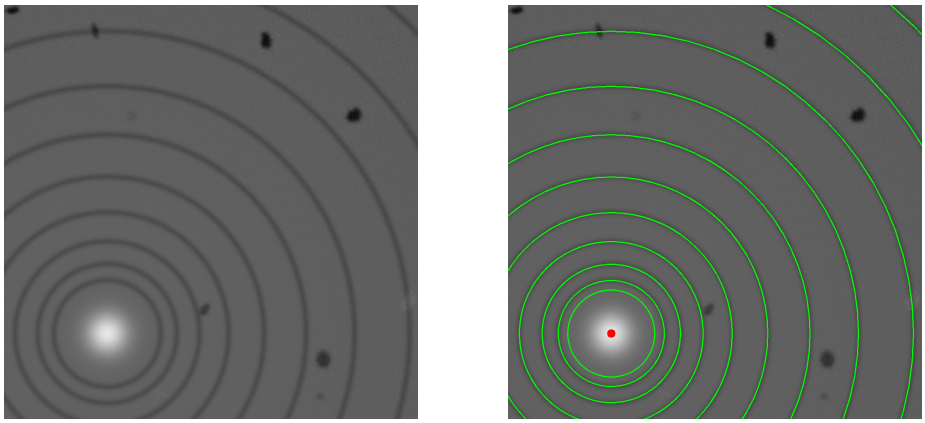
## Contents

**1 Overview** **2**

**2 Brainstorming** **2**

**3 Implementation** **3**
3.1 Center Location . . . . . 4
3.1.1 Contamination . . . . . 4
3.1.2 Noise . . . . . 4
3.2 Fringe Location . . . . . 4
3.2.1 Contamination . . . . . 5
3.2.2 Noise . . . . . 5
3.3 Fringe Radius Regression . . . . . 6

**4 Results** **6**
4.1 Efficiency . . . . . 6
4.2 Limitations . . . . . 9



(a) Example image

(b) Example image, annotated with center (in red) and fringes (in green)

Figure 1: The example image used in the original challenge `README.md`, before and after annotation.

# 1 Overview

In this project, I have implemented a basic code to solve the Metrology Algorithms Image Processing Challenge. The code locates the center and circular fringes within an image (see Figure 1), and uses a regression to estimate a relationship between fringe number and radius.

In this outline, I follow a roughly chronological structure in the development of my code, with the intention of providing some insight into the way I approached the problem. In Section 2, I describe my initial thoughts on how to approach the problem, as well as some of the resources I consulted. In Section 3, I discuss the details of my final implementation. In Section 4, I show examples of my code's results, and analyze its efficiency and limitations.

# 2 Brainstorming

Most of this section does not directly pertain to my final implementation; however it is intended to give some insight on the way I approached the problem.

Coming to this project, I had never used `OpenCV`, so my first step was to familiarize myself with the library. I did this by reading through some of the tutorials available on the official website [2]. As I did so, I kept an eye out for methods to detect circles.

My first idea was to detect the fringes, then use them to find the central gaussian. I believed that this would be more robust to noise and other variations on the problem, since contamination could easily resemble the central Gaussian. So, I primarily focused on ways to find circular shapes.

I read early on about the Hough transform, and this appeared to be an ideal way to solve the problem. I spent a time learning about the method and testing it, with some success. My best attempt involved passing the image through a sharpening filter, then computing the Hough transform to locate circles.

However, the circle detection was unreliable and sensitive to variations in the image. In particular, it had difficulty with the fuzzy edges of the fringes, and with circles that were cut off at the edges of the image. I tried Canny edge detection and `OpenCV` blob detection, with similar results. While I could have addressed these issues (e.g. by extrapolating the image) and moved forward, it became clear that this was not the optimal approach.

The next idea I tried involved taking a Fourier Transform of the image. The clear oscillatory behavior of the solution made it seem like it could be suited to the problem. However, a 2D Cartesian Fourier Transform failed to account for the radial symmetry of the problem, and so of course did not work as intended.

The next logical step was to account for this symmetry using some sort of radial Fourier Transform, but this would require knowing the location of the center. While thinking of solutions to this problem, a new solution struck me: If I knew the center, I could simply use radial brightness as the primary criterion for fringes, rather than worrying about shape detection.

This observation led me to my final implementation, which is described in Section 3.

While coding my solution, my main source of information was the documentation of `OpenCV` [2]. In addition, I made reference to the `NumPy` documentation [1], the `SciPy` documentation [3], and a number of online forum responses for specific questions relevant to my code (which I unfortunately failed to record, and so am unable to cite here).

# 3 Implementation

The general process my implementation uses is as follows:

1. Locate the central white patch on the image.
2. Compute the average radial brightness profile centered at this point.
3. Locate suitable brightness minima in the radial profile, and designate these as the fringes.
4. Use fringe radii to calculate a quadratic regression for fringe radius vs. fringe number.

In the following subsections, I discuss my methods for each step.

## 3.1 Center Location

In a noise- and contamination-free image, a good solution would be to simply locate the brightest pixel in the image and declare it to be the center. Without noise, this works quite well, so I use it in this case.

However, if the brightest pixel is not in the center, then the entire code fails catastrophically (see Figure 5). The fringe-finding process depends on averaging radially about the center, so if it is given the wrong center it is completely unable to locate fringes. Such situations occur frequently with either noise or contamination, so a more robust approach is necessary.

### 3.1.1 Contamination

The first challenge to tackle is contamination. Occasionally, light patches elsewhere in the image exceed the brightness at the center (or, more rarely, a dark patch will cover the center). In either case, the solution is to take advantage of the size difference between contamination patches and the central gaussian.

Before selecting a center point, I pass the image through a median filter. Since there is a stark contrast between contaminated pixels and their uncontaminated neighbors, this filter tends to pick the more "typical" pixel value, thereby reducing the contamination's effect. However, the kernel of the median filter is tuned to be small enough to leave the (larger) central gaussian intact.

### 3.1.2 Noise

Noise poses another issue: Sometimes there will be one or two bright pixels, and sometimes entire swaths of a noisy image will be brighter than the central gaussian.

To solve this, I once again take advantage of the size difference between the noise and central gaussian. First, I apply a box-blurring filter with a kernel larger than the central gaussian, which finds the "background" of the image. Subtracting this background from the original isolates the center: Since the characteristic scale of the noise is on the order of the whole image, it is almost entirely negated.

Having applied filters to combat both contamination and noise, it is now safe to designate the brightest pixel as the center.

## 3.2 Fringe Location

My approach to locating the circular fringes is to take advantage of the problem's radial symmetry. This is a powerful simplifying tool: It collapses the problem from 2D to 1D (it does reduce generality, though; see Section 4.2).

To make this simplification, I compute the Euclidean distance between the center point located in Section 3.1 and all other pixels in the image. I then round down to integer "radii" corresponding roughly to the number of pixel widths separating the points. This rounding allows pixels to be grouped by radius.

I compute the radial brightness profile by averaging the values of all pixels at a given radius, then determine fringe locations by finding the relative brightness minima with respect to radius.

As with locating the center, this method is susceptible to contamination and noise, which I address below.

### 3.2.1 Contamination

When locating fringes, dark contaminations pose the greatest problem. The radial brightness profile described above takes no account of whether a local minimum is due to a fringe spread out across the entire radius or due to a single dark contamination, so these dark patches can cause a fringe to be detected where there is none.

I combat this by removing outliers from each radius's profile before computing the average. The contaminations may be dark enough to lower the overall *average*, but their contribution is localized within relatively few pixels, whose deviation from the mean is high. So, I compute the standard deviation of pixel values at each radius, then remove outliers that lie beyond $1\sigma$ from the mean.

***Aside: Bugs***

*While writing this code, I encountered an integer overflow bug. `OpenCV` encodes images with 8-bit integers, so when I squared pixel values to calculate the standard deviation, they all hit the integer maximum. Since they (almost) all had the same value, their standard deviation was 0, causing all points to be treated as outliers and removed. I had a hard time pinpointing the bug, but the solution was easy: Just convert to a 32-bit integer.*

*I later encountered a second strange bug to do with the standard deviation: Whereas my code had previously worked fine on noise/contamination-free images, it now failed. It turns out that this was due to the standard deviation check: With a noise-free image, all pixel values at a given radius are approximately the same, making go to 0. So, again, nearly all points were excluded when removing outliers, resulting in misbehavior. I fixed it by adding a global option to turn on or off noise-resiliency.*

### 3.2.2 Noise

Noise in the image can cause artificial local minima in the brightness profile, resulting in the detection of false fringes. My solution to this is to filter the image with a small box blur, reducing local noise.

In addition, radii with very few contributing pixels are highly susceptible to noise. This occurs at the lowest and highest radii. To fix this, I simply put a hard lower and upper limit on the allowed radii for fringes, beyond which none can be detected. This can cause fringes in the farmost corner of the image to fail to be detected; however, without this check almost all noisy images detect phantom fringes.

### 3.3 Fringe Radius Regression

I run a quadratic regression on the fringe radii to produce a relation between the radius $r$ (in pixel widths) and the number $n$ (counting outward from the center and starting with $n = 0$) of the fringes:

$$r(n) = an^2 + bn + c,$$

where $a$, $b$, $c$ are the parameters determined by the regression.

This is quite accurate to the data if the fringes are detected correctly. However, if a fringe fails to be detected, or a nonexistent fringe is detected, the fit can be thrown off. That being said, I do not do any data cleaning while computing the regression, since the fringe detection code should ideally minimize the possibility of fringe detection errors on its own.

## 4 Results

In Figures 2-5, I have included some examples of circle detection.

Figure 2 is the basic example, with no noise or contaminations. It does not use the noise/contamination mitigation techniques described above, and still does fine. The code makes very few mistakes in this situation.

Figure 3 uses the challenge's recommended noise and contamination levels. The detector used for this figure utilized the noise/contamination mitigation techniques (this is not absolutely necessary for this level of noise, but greatly improves accuracy). The code makes few mistakes, but on rare occasions will miss a fringe or detect one that is nonexistent.

Figures 4 and 5 are examples using 10 times the challenge's recommended noise and contamination levels. Needless to say, they use the noise/contamination mitigation techniques: My goal in choosing such high values was to push the limits of my code. The code makes more minor mistakes (e.g. missed or hallucinated fringes) in this situation, but still few complete errors (as in Figure 5).

### 4.1 Efficiency

Almost all parts of the code run in $\mathcal{O}(N)$ time, where $N$ is the number of pixels in the image (neglecting contributions from different numbers of radii $R$, since $R \ll N$). However, one function call pushes the entire code to $\mathcal{O}(N \log N)$ time: `numpy.unique()`.

I am confident there is a way to rewrite this in order to bring the overall code down to $\mathcal{O}(N)$; however I have focused this prototype code more on accuracy and resilience, so I have left this as it stands for the present.

Using the images from `metalg_challenge_compiled.generate_image()`, my code consistently takes around $60 \pm 20$ ms to run without noise-resilience, or $120 \pm 20$ ms with it.[1]

[1] On my hardware: 2017 MacBook Pro, 2.3 GHz Dual-Core Intel Core i5, 8 GB RAM

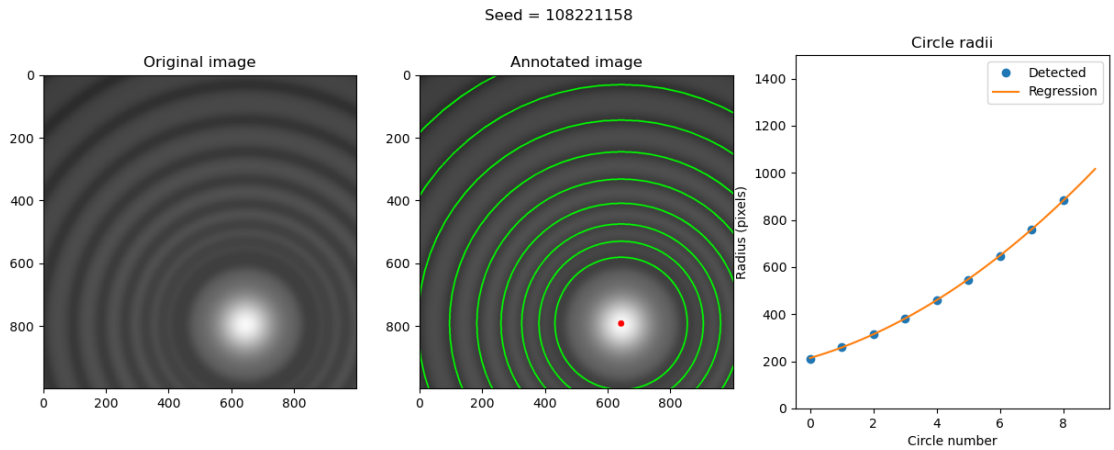



Figure 2: Circle detection with no noise or contamination. Seed: 108221158.

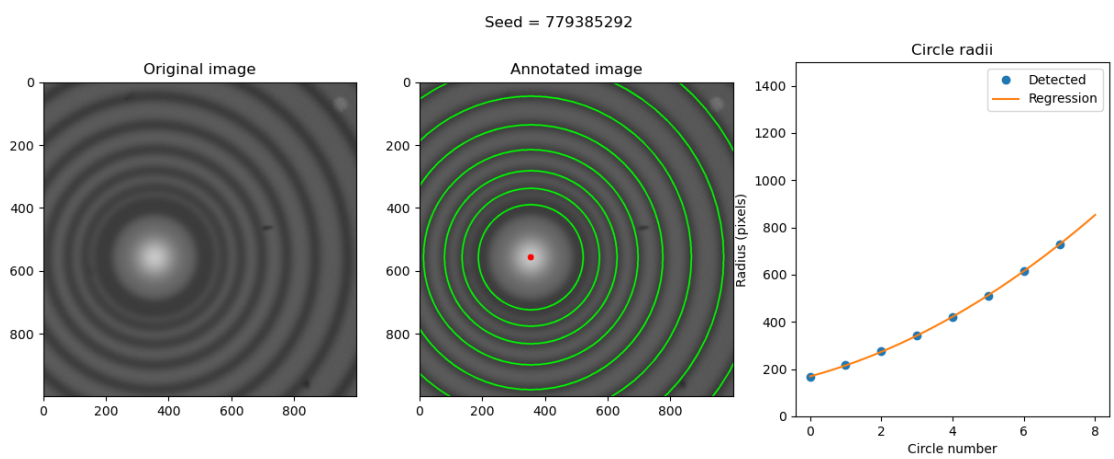



Figure 3: Circle detection with middling noise and contamination. Number of contaminations: 5, Noise magnitude: 10, Seed: 779385292.

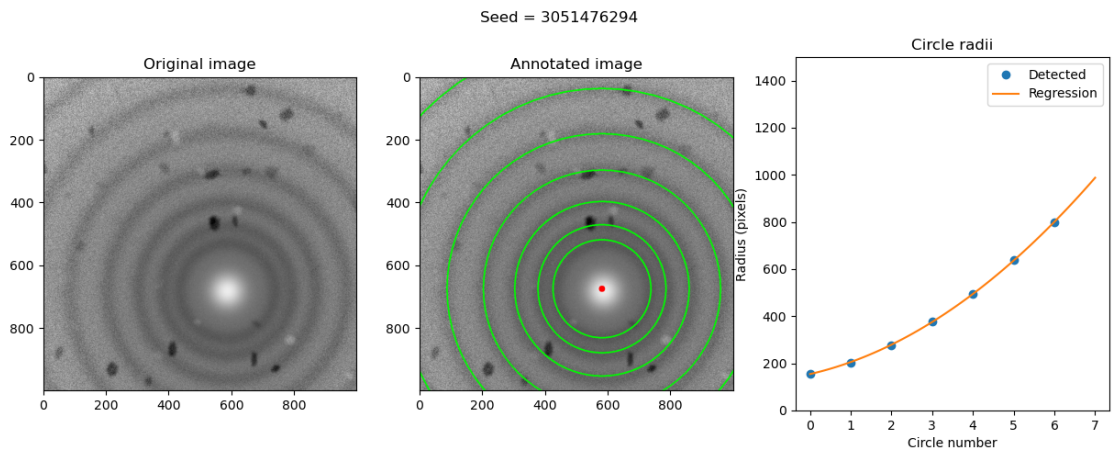



Figure 4: Circle detection with high noise and contamination. Number of contaminations: 50, Noise magnitude: 100, Seed: 3051476294.

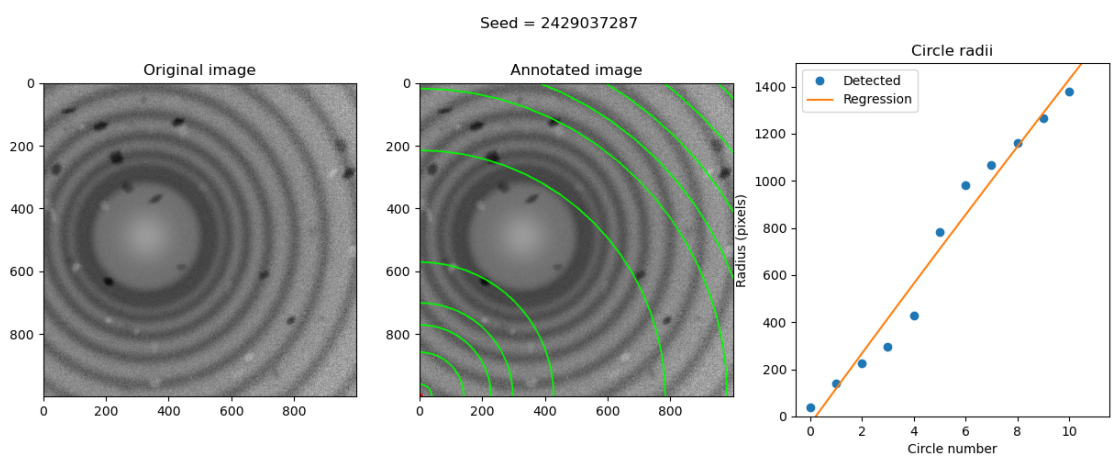



Figure 5: Failed circle detection. Here, the center was chosen incorrectly. Number of contaminations: 50, Noise magnitude: 100, Seed: 2429037287.

### 4.2 Limitations

My solution to this code takes advantage of several aspects specific to the images. Doing so allows my code to work well in this specific case; however it also makes it more difficult to generalize.

The largest constraint is that of radial symmetry. My code works only on images which consist of perfectly concentric rings about the center. If they do not fit this shape, then my radial averaging technique fails.

One instance of where this could be problematic is as follows: If a real-world detector creating these images were slightly tilted, perspective could cause the circles to appear elliptical, causing failure. A potential solution for this would be to use some image processing technique to detect if the image appeared stretched, then correct for this to create concentric circles.

Another constraint comes from the brightness techniques I use. If, for instance, both the center and the fringes were bright, my code would similarly fail (although this specific case could be addresed by simply detecting extrema of the brightness profile).

A final important constraint is the tuned parameters. In my code, I have chosen specific numerical parameters (e.g kernel widths, maximum permitted deviations from the mean) that work for the images provided. If the images provided to the code were, say, zoomed in or out, the code would be likely to fail. For more general use cases, it would be a good idea to make these parameters adjustable.

## References


[1] NumPy Reference. `https://numpy.org/doc/2.4/reference/index.html#reference`. [Online; accessed 18-March-2026].

[2] OpenCV - Python Tutorials. `https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html`. [Online; accessed 18-March-2026].

[3] SciPy API. `https://docs.scipy.org/doc//scipy/reference/index.html#scipy-api`. [Online; accessed 18-March-2026].